BÀI LUYỆN TẬP ABSTRACT CLASS

Bài 1: Tạo lớp trừu tượng `Shape`

Đề bài

Tạo một lớp trừu tượng `Shape` có:

- Thuộc tính: `color` (String).
- Phương thức trừu tượng: `double tinhDienTich()`, `double tinhChuVi()`.
- Lớp con `Rectangle` (hình chữ nhật) có thuộc tính `width`, `height` và triển khai phương thức tính diện tích, chu vi.
- Lớp con `Circle` (hình tròn) có thuộc tính `radius` và triển khai phương thức tính diện tích, chu vi.

Giải pháp

```java
// Abstract class
abstract class Shape {
    protected String color;

    public Shape(String color) {
        this.color = color;
    }

    // Phương thức trừu tượng
    public abstract double tinhDienTich();
    public abstract double tinhChuVi();
}

// Lớp Rectangle kế thừa Shape
class Rectangle extends Shape {
    private double width;
    private double height;

    public Rectangle(String color, double width, double height) {
        super(color);
        this.width = width;
        this.height = height;
    }

    @Override
    public double tinhDienTich() {
        return width * height;
    }

    @Override
    public double tinhChuVi() {
        return 2 * (width + height);
    }
}
```

```java
// Lớp Circle kế thừa Shape
class Circle extends Shape {
    private double radius;

    public Circle(String color, double radius) {
        super(color);
        this.radius = radius;
    }

    @Override
    public double tinhDienTich() {
        return Math.PI * radius * radius;
    }

    @Override
    public double tinhChuVi() {
        return 2 * Math.PI * radius;
    }
}

public class Main {
    public static void main(String[] args) {
        Shape hcn = new Rectangle("Red", 5, 3);
        Shape htron = new Circle("Blue", 4);

        System.out.println("Diện tích hình chữ nhật: " + hcn.tinhDienTich());
        System.out.println("Chu vi hình chữ nhật: " + hcn.tinhChuVi());

        System.out.println("Diện tích hình tròn: " + htron.tinhDienTich());
        System.out.println("Chu vi hình tròn: " + htron.tinhChuVi());
    }
}
```

Phân tích

- **Abstract class `Shape`** chứa phương thức trừu tượng để các lớp con phải triển khai.
- **Lớp `Rectangle` và `Circle`** kế thừa từ `Shape` và thực hiện các phương thức cụ thể.
- **Tính đa hình**: Dùng biến `Shape` để tham chiếu tới các đối tượng `Rectangle` và `Circle`.

Bài 2: Xây dựng lớp `Animal`

Đề bài

Tạo lớp trừu tượng `Animal` với:

- Phương thức trừu tượng `makeSound()`.
- Lớp con `Dog` và `Cat` kế thừa `Animal`, triển khai phương thức `makeSound()`.

Giải pháp

```java
// Abstract class
abstract class Animal {
    public abstract void makeSound();
}

// Lớp Dog kế thừa Animal
class Dog extends Animal {
    @Override
    public void makeSound() {
        System.out.println("Woof woof!");
    }
}

// Lớp Cat kế thừa Animal
class Cat extends Animal {
    @Override
    public void makeSound() {
        System.out.println("Meow meow!");
    }
}

public class Main {
    public static void main(String[] args) {
        Animal myDog = new Dog();
        Animal myCat = new Cat();

        myDog.makeSound();
        myCat.makeSound();
    }
}
```

Phân tích

- **Abstract class `Animal`** có phương thức trừu tượng `makeSound()`, bắt buộc các lớp con triển khai.
- **Lớp `Dog` và `Cat`** định nghĩa cách phát âm thanh của riêng mình.
- **Đa hình**: Có thể dùng `Animal` để tham chiếu đến `Dog` hoặc `Cat`.

Bài 3: Quản lý phương tiện giao thông

Đề bài

Tạo lớp trừu tượng `Vehicle` với:

- Thuộc tính: `brand`, `year`.
- Phương thức trừu tượng: `start()`, `stop()`.
- Lớp con `Car` và `Motorcycle` kế thừa `Vehicle`, triển khai phương thức `start()` và `stop()`.

Giải pháp

```java
// Abstract class
abstract class Vehicle {
    protected String brand;
    protected int year;

    public Vehicle(String brand, int year) {
        this.brand = brand;
        this.year = year;
    }

    public abstract void start();
    public abstract void stop();
}

// Lớp Car kế thừa Vehicle
class Car extends Vehicle {
    public Car(String brand, int year) {
        super(brand, year);
    }

    @Override
    public void start() {
        System.out.println(brand + " Car is starting...");
    }

    @Override
    public void stop() {
        System.out.println(brand + " Car is stopping...");
    }
}

// Lớp Motorcycle kế thừa Vehicle
class Motorcycle extends Vehicle {
    public Motorcycle(String brand, int year) {
        super(brand, year);
    }

    @Override
    public void start() {
        System.out.println(brand + " Motorcycle is starting...");
    }

    @Override
    public void stop() {
        System.out.println(brand + " Motorcycle is stopping...");
    }
}

public class Main {
    public static void main(String[] args) {
        Vehicle myCar = new Car("Toyota", 2022);
        Vehicle myMotorcycle = new Motorcycle("Honda", 2021);

        myCar.start();
        myCar.stop();

        myMotorcycle.start();
```

```
        myMotorcycle.stop();
    }
}
```

Phân tích

- **Abstract class `Vehicle`** chứa phương thức trừu tượng `start()` và `stop()`.
- **Lớp `Car` và `Motorcycle`** triển khai cách khởi động và dừng lại của riêng chúng.
- **Tính đa hình** giúp gọi `start()` và `stop()` trên `Vehicle`.

Bài 4: Xây dựng hệ thống nhân viên

Đề bài

Tạo lớp trừu tượng `Employee` với:

- Thuộc tính: `name`, `salary`.
- Phương thức trừu tượng: `calculateBonus()`.
- Lớp con `Manager` và `Developer` kế thừa `Employee`, triển khai phương thức `calculateBonus()`.

Giải pháp

```
// Abstract class
abstract class Employee {
    protected String name;
    protected double salary;

    public Employee(String name, double salary) {
        this.name = name;
        this.salary = salary;
    }

    public abstract double calculateBonus();
}

// Lớp Manager kế thừa Employee
class Manager extends Employee {
    public Manager(String name, double salary) {
        super(name, salary);
    }

    @Override
    public double calculateBonus() {
        return salary * 0.2; // Bonus 20%
    }
}

// Lớp Developer kế thừa Employee
class Developer extends Employee {
    public Developer(String name, double salary) {
```

```java
        super(name, salary);
    }

    @Override
    public double calculateBonus() {
        return salary * 0.1; // Bonus 10%
    }
}

public class Main {
    public static void main(String[] args) {
        Employee manager = new Manager("Alice", 5000);
        Employee developer = new Developer("Bob", 4000);

        System.out.println("Manager bonus: " + manager.calculateBonus());
        System.out.println("Developer bonus: " + developer.calculateBonus());
    }
}
```

Phân tích

- **Abstract class `Employee`** buộc các lớp con phải triển khai phương thức `calculateBonus()`.
- **`Manager` và `Developer`** có cách tính thưởng khác nhau.
- **Tính đa hình** giúp gọi `calculateBonus()` mà không cần biết kiểu cụ thể.

Tổng kết

- **Abstract class** giúp định nghĩa các phương thức chung mà các lớp con phải triển khai.
- **Đa hình** giúp linh hoạt khi làm việc với các đối tượng kế thừa.
- **Tách biệt trách nhiệm** giữa các lớp cha và lớp con.

Bạn có thể thử mở rộng các bài toán này bằng cách thêm nhiều thuộc tính và phương thức khác

BÀI LUYỆN TẬP THÊM

Hướng dẫn luyện tập

- **Nhóm 1:** Luyện tập về cách khai báo **abstract class** và triển khai trong lớp con.
- **Nhóm 2:** Áp dụng **abstract class** vào thực tế, sử dụng **ArrayList** để quản lý danh sách đối tượng.

Nhóm 1: Cơ bản – Tạo lớp trừu tượng và kế thừa

1. **Tạo lớp trừu tượng `Animal` với phương thức `makeSound()`, lớp con `Dog` và `Cat` triển khai phương thức.**
2. **Tạo lớp trừu tượng `Shape` có phương thức `tinhDienTich()` và `tinhChuVi()`, lớp con `Rectangle` và `Circle` kế thừa và triển khai.**
3. **Tạo lớp trừu tượng `Vehicle` có phương thức `start()` và `stop()`, lớp con `Car` và `Motorcycle` kế thừa.**
4. **Tạo lớp trừu tượng `Employee` có phương thức `calculateSalary()`, lớp con `Manager` và `Developer` triển khai.**
5. **Tạo lớp trừu tượng `Payment` với phương thức `processPayment()`, lớp con `CreditCardPayment` và `PaypalPayment` kế thừa và triển khai.**
6. **Tạo lớp trừu tượng `Appliance` với phương thức `turnOn()` và `turnOff()`, lớp con `WashingMachine` và `Refrigerator` kế thừa.**
7. **Tạo lớp trừu tượng `Account` có phương thức `withdraw()` và `deposit()`, lớp con `SavingsAccount` và `CurrentAccount` kế thừa và triển khai.**
8. **Tạo lớp trừu tượng `Instrument` với phương thức `playSound()`, lớp con `Guitar` và `Piano` triển khai.**
9. **Tạo lớp trừu tượng `Document` với phương thức `printContent()`, lớp con `WordDocument` và `PDFDocument` kế thừa.**
10. **Tạo lớp trừu tượng `Person` có phương thức `getRole()`, lớp con `Teacher` và `Student` kế thừa và triển khai.**

Nhóm 2: Nâng cao – Xử lý danh sách đối tượng, mở rộng lớp

11. **Quản lý danh sách phương tiện (`Vehicle`) với danh sách `Car`, `Bike` và in ra phương tiện có tốc độ cao nhất.**
12. **Quản lý danh sách nhân viên (`Employee`), tính tổng lương của tất cả nhân viên.**
13. **Tạo hệ thống đăng nhập với lớp `User` (trừu tượng) và lớp con `Admin`, `Customer` để phân quyền truy cập.**
14. **Quản lý kho sản phẩm (`Product`) với danh sách `Electronics`, `Furniture`, tính tổng giá trị kho hàng.**
15. **Tạo hệ thống quản lý giao dịch với lớp `Transaction` (trừu tượng) và lớp con `OnlinePayment`, `CashPayment`.**
16. **Xây dựng hệ thống thú cưng (`Pet`) với các lớp con `Dog`, `Cat`, `Bird`, cho phép hiển thị tiếng kêu của từng loại thú cưng.**
17. **Tạo hệ thống khách sạn (`HotelRoom`) với các loại phòng `StandardRoom`, `DeluxeRoom`, tính tổng doanh thu dự kiến.**
18. **Xây dựng hệ thống kiểm soát động vật (`WildAnimal`, `DomesticAnimal`) và hiển thị đặc điểm của từng nhóm.**
19. **Quản lý thư viện (`LibraryItem`), các lớp con `Book`, `Magazine` kế thừa và quản lý danh sách tài liệu.**
20. **Tạo hệ thống đặt vé (`Ticket`), lớp con `BusTicket`, `TrainTicket`, quản lý danh sách vé đã đặt.**